

전자전기컴퓨터설계실험Ⅱ

LAB 1. VS Code Verilog

환경설정 매뉴얼

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

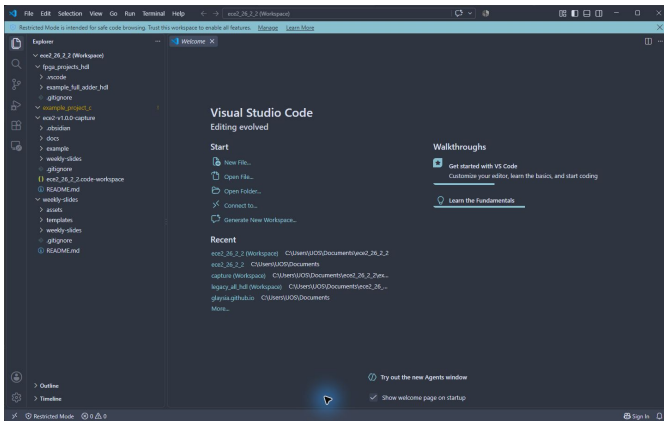
설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

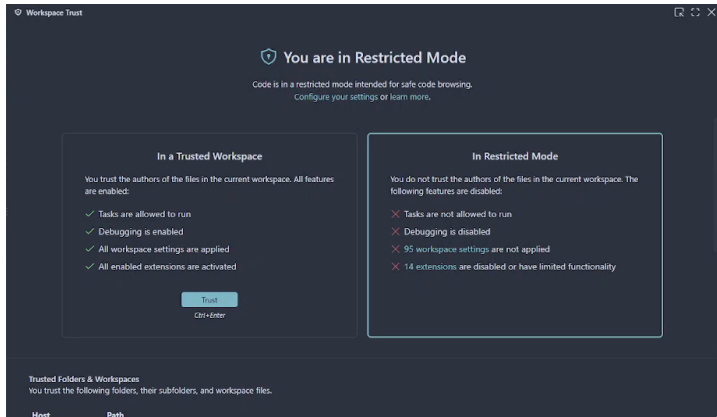
- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

3. 실제 화면 : 제한 모드 확인



상단 안내와 왼쪽 아래 Restricted Mode를 확인한다.

3. 실제 화면 : Trust 선택



수업 저장소임을 확인한 뒤 왼쪽 Trust를 선택한다.



설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

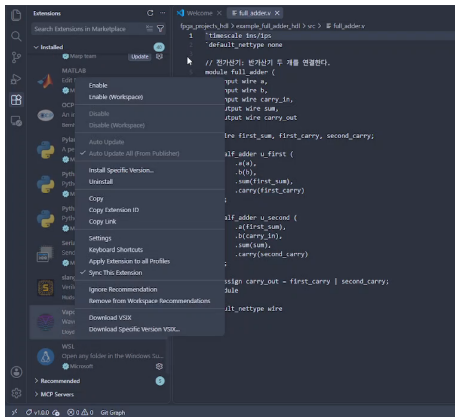
설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

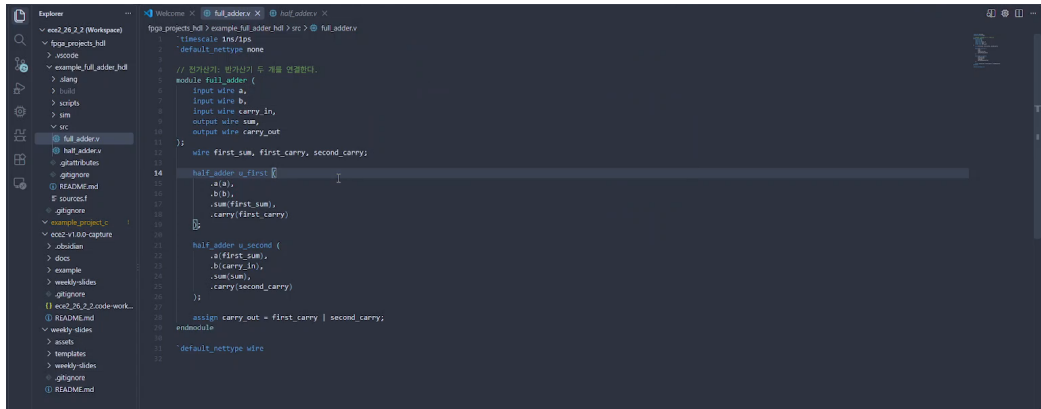
- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

4. 실제 화면 : Enable (Workspace)



확장을 설치했어도 현재 워크스페이스에서 꺼져 있으면 활성화한다.

5. 실제 화면 : 전가산기 코드



```
1 timescale 1ns/1ps
2 `default_nettype none
3
4 // 전가산기: 반가산기 두 개를 연결한다.
5 module full_adder (
6     input wire a,
7     input wire b,
8     input wire carry_in,
9     output wire sum,
10    output wire carry_out
11);
12    wire first_sum, first_carry, second_carry;
13
14    half_adder u_first (
15        .a(a),
16        .b(b),
17        .sum(first_sum),
18        .carry(first_carry)
19    );
20
21    half_adder u_second (
22        .a(first_sum),
23        .b(carry_in),
24        .sum(sum),
25        .carry(second_carry)
26    );
27
28    assign carry_out = first_carry | second_carry;
29 endmodule
30
31 `default_nettype wire
32
```

반가산기 두 개의 연결을 확인한다. 색상 표시만으로 분석 성공을 판단하지 않는다.



설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

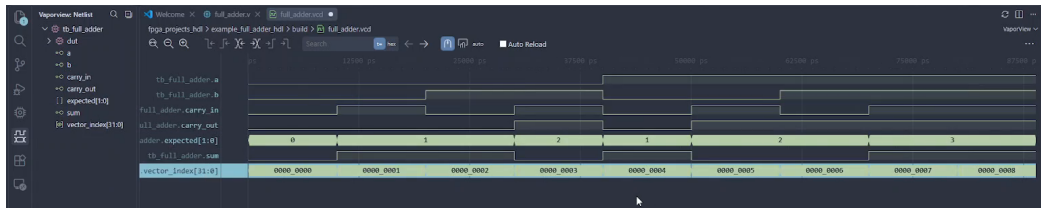
설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

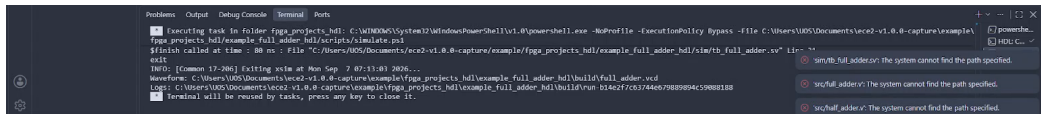
- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

8. 실제 화면 : 8개 입력의 파형



시간 단위는 ps이다. 10,000 ps마다 입력이 바뀌며 80 ns까지 비교한다.

8. 실제 화면 : 실행 완료와 별도 오류 구분



- 터미널에 80 ns 종료와 생성된 VCD 경로가 표시된다.
- 오른쪽 소스 경로 오류 알림은 파형 생성 결과와 구분한다.
- 소스 분석·이동 문제가 남으면 언어 서버 설정의 상대경로 기준을 확인한다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.



설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.



설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.

설치 완료 확인

- Git·VS Code의 버전과 경로를 확인했다. GitHub CLI는 사용 시 확인했다.
- Vivado 도구 세 개가 2026.1로 실행된다.
- 올바른 예제 폴더를 열고 두 확장을 설치했다.
- 정의 이동과 오류 진단을 확인하고 문법 오류를 복원했다.
- 8개 입력 검사 PASS와 80 ns 파형을 확인했다.
- 다섯 신호의 값을 진리표와 비교하여 설명할 수 있다.